



Digital Transformation

PDD — PROCESS DEFINITION DOCUMENT

Processo de Desenvolvimento com GitFlow — Portal Fake

Técnicas de Hyperautomation

Janaina dos Santos Ferreira | Prof. Moisés Levy

Versão: 1.0 | Data: 07/07/2026

Histórico de Revisões

Versão	Data	Responsável	Descrição da Alteração	Status
1.0	07/07/2026	Janaina S. Ferreira	Criação inicial do documento	Aprovado



Digital Transformation

Sumário

Histórico de Revisões	1
1 Descrição do Processo	3
2 Objetivo do Processo	3
3 Escopo do Processo	3
4 Responsável pelo Processo	3
5 Fluxo do Processo (Resumo)	3
6 Regras de Negócio	4
7 Pré-condições	4
8 Ambiente de Desenvolvimento	4
9 Estrutura do Processo (Resumo Técnico)	5
10 Diagrama do Fluxo GitFlow	5
11 Entradas e Saídas	5
12 Exceções	6
13 Comandos Git — Simulação do Fluxo GitFlow	6
13.1 Módulo de Cadastro	6
13.2 Módulo de Consulta	6
13.3 Módulo de Exportação	6
13.4 Validação (QA) e Merge Final	7
14 Viabilidade da Automação/Padronização	7
15 Observações Finais	7

1 Descrição do Processo

O processo descrito neste documento trata do fluxo de desenvolvimento colaborativo de software aplicado ao **Portal Fake**, sistema utilizado como ambiente controlado para prática de automação (RPA) e Hyperautomation.

O Portal Fake é tratado, neste contexto, como um sistema em evolução contínua, cujas funcionalidades são desenvolvidas por diferentes integrantes da equipe de forma paralela e organizada. Para viabilizar esse trabalho colaborativo sem conflitos e com rastreabilidade total das alterações, adota-se o **GitFlow** como modelo de controle de versão, em conjunto com o **Git** e o **GitHub**.

2 Objetivo do Processo

O objetivo deste processo é padronizar e documentar o fluxo de desenvolvimento colaborativo do Portal Fake, garantindo que:

- cada funcionalidade seja desenvolvida de forma isolada, sem impactar o código estável;
- todas as alterações passem por revisão antes de serem integradas;
- o histórico de mudanças seja rastreável e reversível;
- o versionamento reflita, de forma organizada, a evolução do sistema.

3 Escopo do Processo

Início: Criação de uma *feature branch* a partir da branch *develop* para o desenvolvimento de um módulo específico.

Fim: Merge da funcionalidade concluída e aprovada na branch *main*, tornando-a parte da versão estável do sistema.

4 Responsável pelo Processo

- **Desenvolvedor(a) responsável pelo Cadastro:** implementação do módulo `cadastro.py`.
- **Desenvolvedor(a) responsável pela Consulta:** implementação do módulo `consulta.py`.
- **Desenvolvedor(a) responsável pela Exportação:** implementação do módulo `exportacao.py`.
- **QA (Validação e Testes):** responsável por validar as funcionalidades antes da aprovação do Pull Request.
- **Revisor(a)/Aprovador(a):** responsável pela revisão de código (code review) e aprovação do merge.

5 Fluxo do Processo (Resumo)

O fluxo de desenvolvimento segue as etapas abaixo:

1. Criação do repositório no GitHub.
2. Criação da branch `develop` a partir da `main`.
3. Criação das *feature branches* (`feature/cadastro`, `feature/consulta`, `feature/exportacao`).
4. Desenvolvimento dos módulos correspondentes por cada integrante.
5. Commit e push das alterações para o repositório remoto.
6. Abertura de Pull Request (PR) da *feature branch* para a `develop`.
7. Revisão de código (code review) pelo(a) responsável.
8. Aprovação e merge na branch `develop`.
9. Validação do QA na branch `develop`.
10. Merge final da `develop` para a `main`, consolidando a versão estável.

6 Regras de Negócio

- Não é permitido desenvolver diretamente na branch `main`.
- Cada integrante deve trabalhar exclusivamente em sua *feature branch*.
- A abertura de Pull Request é obrigatória para qualquer integração.
- O merge só é realizado mediante aprovação do(a) revisor(a).
- Correções solicitadas na revisão retornam para a *feature branch* de origem.
- Mensagens de commit devem seguir um padrão descritivo (ex.: `feat: implementar módulo de cadastro`).

7 Pré-condições

- Git instalado na máquina do desenvolvedor.
- Conta e repositório configurados no GitHub.
- Repositório do Portal Fake criado e acessível à equipe.
- Equipe definida, com módulos previamente distribuídos.
- Ambiente de desenvolvimento (VS Code) configurado.

8 Ambiente de Desenvolvimento

O processo de desenvolvimento do Portal Fake é realizado em ambiente colaborativo, com o uso das seguintes ferramentas:

- **Git:** sistema de controle de versão distribuído, utilizado para o gerenciamento do código-fonte.
- **GitHub:** plataforma de hospedagem do repositório e controle colaborativo do projeto.
- **Visual Studio Code (VS Code):** editor utilizado para o desenvolvimento das funcionalidades.
- **Python:** linguagem utilizada na implementação dos módulos do Portal Fake.

9 Estrutura do Processo (Resumo Técnico)

O repositório segue a organização de branches definida pelo GitFlow:

- main → versão estável do sistema.
- develop → branch de integração das funcionalidades.
- feature/* → branches de desenvolvimento de novas funcionalidades (feature/cadastro, feature/consulta, feature/exportacao).

Resumo técnico do fluxo:

- Versionamento com Git.
- Organização em branches conforme o GitFlow.
- Integração via Pull Request.
- Revisão de código (code review).
- Merge controlado para develop e main.

10 Diagrama do Fluxo GitFlow

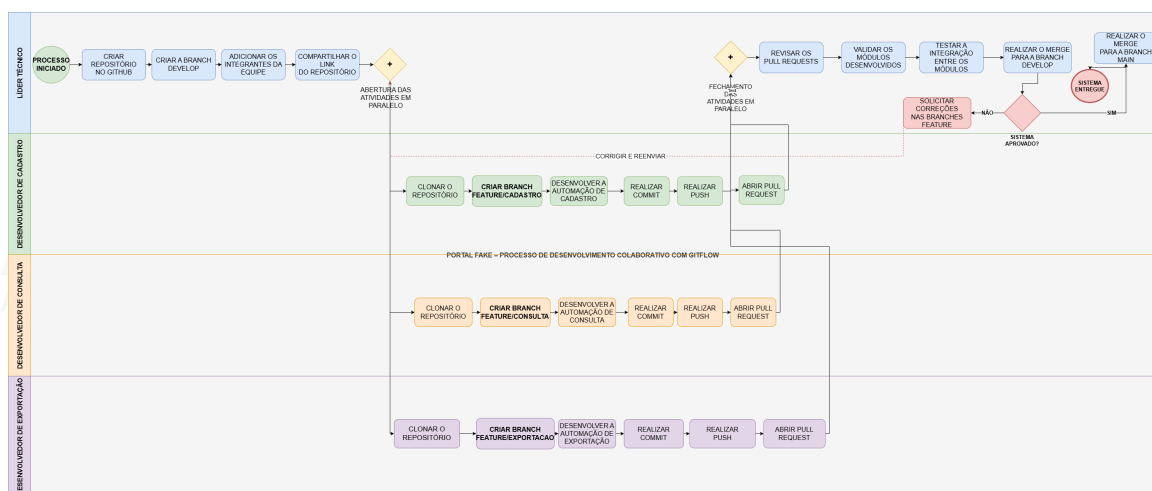


Figura 1: Fluxo de branches do Processo de Desenvolvimento com GitFlow — Portal Fake

11 Entradas e Saídas

Entradas: Código-fonte dos módulos (cadastro.py, consulta.py, exportacao.py), commits, Pull Requests.

Saídas:

- Branches develop e main atualizadas.
- Histórico de commits e Pull Requests registrado no GitHub.
- Versão estável e testada do Portal Fake.

12 Exceções

- Pull Request rejeitado.
- Conflito de merge entre branches.
- Erro no código da automação identificado na revisão.
- Falha na validação do sistema pelo QA.

13 Comandos Git — Simulação do Fluxo GitFlow

Para reproduzir o processo descrito, cada integrante executa a sequência de comandos abaixo em sua respectiva *feature branch*.

13.1 Módulo de Cadastro

```
git checkout develop
git pull origin develop
git checkout -b feature/cadastro

# desenvolvimento do modulo cadastro.py

git add .
git commit -m "feat: implementar modulo de cadastro"
git push origin feature/cadastro
# Abrir Pull Request: feature/cadastro -> develop
# Revisar, aprovar e realizar o merge na develop
```

13.2 Módulo de Consulta

```
git checkout develop
git pull origin develop
git checkout -b feature/consulta

# desenvolvimento do modulo consulta.py

git add .
git commit -m "feat: implementar modulo de consulta"
git push origin feature/consulta
# Abrir Pull Request: feature/consulta -> develop
# Revisar, aprovar e realizar o merge na develop
```

13.3 Módulo de Exportação

```
git checkout develop
git pull origin develop
git checkout -b feature/exportacao

# desenvolvimento do modulo exportacao.py

git add .
git commit -m "feat: implementar modulo de exportacao"
git push origin feature/exportacao
```

```
# Abrir Pull Request: feature/exportacao -> develop
# Revisar, aprovar e realizar o merge na develop
```

13.4 Validação (QA) e Merge Final

```
git checkout develop
git pull origin develop
# validacao e testes do QA

git checkout main
git pull origin main
git merge develop
git push origin main
```

Atenção

Se um Pull Request for rejeitado ou houver conflito de merge, a correção deve ser feita na *feature branch* de origem e reenviada:

```
git checkout feature/<nome-do-modulo>
# corrigir o código

git add .
git commit -m "fix: corrigir conforme revisao"
git push origin feature/<nome-do-modulo>
# abrir Pull Request novamente
```

14 Viabilidade da Automação/Padronização

(X) Sim

Justificativa: Processo estruturado, repetitivo a cada nova funcionalidade, com regras claras de branches, revisão e integração — características que favorecem a padronização documental via PDD.

15 Observações Finais

O processo aqui documentado representa um cenário simulado de desenvolvimento colaborativo, utilizando o Portal Fake como ambiente de prática. A adoção do GitFlow, combinada ao uso do Git e do GitHub, permite reproduzir, em escala reduzida, práticas reais de controle de versão e integração de código utilizadas em ambientes profissionais de desenvolvimento de software.

fim do capítulo ■



Digital Transformation